Niagara 4 point deployment · Part 1 of 2

Publishing Station Points

The station as a server: BACnet/IP, oBIX, Modbus TCP slave

Verified on Niagara 4.14.0.162 · every object read back by an independent client

Part 2 covers the opposite direction: building clients that collect these points

Shanghai Gline Net Co., Ltd.

@jason912 · jason.zhang@gline-net.com

One pattern, three protocols

Publishing a point is never one operation the binding is the second

Step	BACnet export	oBIX export	Modbus TCP slave
1 · Create the object	REST add□bacnet:*Descriptor	REST add□obixDriver:ObixExport	REST add with PROXY□(or cp from a built point)
2 · Bind source / address	pointOrd oBIX PUT	point oBIX PUT	dataAddress REST mod
3 · Public identity	objectId REST mod	binding is the identity	binding is the identity
Channel count	REST + oBIX	REST + oBIX	REST only

Why step 2 decides how many channels you need

A source binding is a BOrd slot. REST creates and edits the object; the ord itself goes in over oBIX, so a descriptor or export is a two-channel build.

A Modbus slave point carries its binding as a proxy extension, so the whole build stays inside REST.

Assign the identity slot in step 3 and read it back the object name follows the identifier you wrote.

Where the objects live

Three protocols, three trees and only one of them is a single flat table

BACnet/IP

```
/Drivers/BacnetNetwork
  bacnetComm/network/ipPort
    link  (adapter    )
  localDevice
    exportTable
      <descriptor>     1 point
```

oBIX

```
/Drivers/ObixNetwork
  server   (servlet)
  exports
    <export>     1 point

same /obix/config tree
```

Modbus TCP slave

```
/Drivers/<slaveNetwork>
  port  (ServerPort)
  <slaveDevice>
    points
      <point>
        dataAddress
```

BACnet is the only protocol with a separate network-enable layer above the objects: the network, the port and the link must all be up before a descriptor can answer on the wire. oBIX and the Modbus slave listen as soon as their own object is enabled.

What the trees imply for a build order

BACnet: enable top-down. The network, the port and the link are three separate switches, and the descriptor answers once all three are on.

oBIX: the servlet and the export folder are siblings one is the door, the other is the list, and neither depends on the other being edited.

Modbus slave: the port belongs to the network, so one port serves every slave device under it.

BACnet/IP: three enable layers before anything answers

Enabling only the port leaves it {ok} while nothing replies on UDP 47808

Layer	Path	Working read-back
Driver network	/Drivers/BacnetNetwork	enabled = true, status {ok}
Network port	/bacnetComm/network/ipPort	enabled = true, status {ok}
Link	/ipPort/link	adapter resolves to the physical NIC (eth6)

The link holds three enum slots

adapter, adapterId and ipAddress are DynamicEnum slots. REST rejects them twice BEnumRange cannot be cast to BEnum for the value, Illegal name for the display text. The one working channel is an oBIX PUT of the escaped name read from the slot's ~range.

```
GET  /ipPort/link/adapter/~range
PUT  /ipPort/link/adapter/  <enum val="Intel$28R$29$20PRO$2f1000$20MT$20Desktop$20Adapter$20$232"/>
  adapterId and ipAddress follow automatically (eth6 / the station's own address)
```

A virtual or debug adapter is a common wrong value and produces adapter not ready. Completion is four green states at once: network {ok}, ipPort {ok}, a clean status line, localDevice Operational.

BACnet export table: one descriptor per point

Three slots, two channels and a component that renames itself

Slot	Type	Channel	Rule
objectName	String	REST mod	unique on the whole network
pointOrd	BOrd	oBIX PUT only	one source point backs one descriptor
objectId	BBacnetObjectIdentifier	REST mod	e.g. analogValue:100

Descriptor type follows the source point

Source	Descriptor
control:NumericPoint (read-only)	BacnetAnalogInputDescriptor
Numeric, value / output semantics	BacnetAnalogValue / AnalogOutputDescriptor
control:BooleanPoint	BacnetBinaryInputDescriptor
control:EnumPoint	BacnetEnumDescriptor

Writing objectId renames the component: demo_av100 becomes analogValue_100. Look it up by listing the table, never by the name you typed.

A # in the source ord must be escaped as $23, and one source point backs exactly one descriptor.

pointOrd can answer BadUriErr and still have taken effect confirm by reading the slot back, not by the response code.

oBIX export: the binding only oBIX can write

The same BOrd limit, seen from the other side

Create over REST, bind over oBIX

add obixDriver:ObixExport not obix:ObixExport, which answers type-not-found then PUT the source ord into the point slot.

```
PUT  /obix/config/Drivers/ObixNetwork/exports/<name>/point/
<uri val="station:|slot:/<path>/<point>"/>    #  # in the path becomes $23
```

Writing through an export

POST to the export's set node, not to the export object. A proxied source point accepts the write and does not move that is a mapping limit, not an authentication problem.

```
POST /obix/config/Drivers/ObixNetwork/exports/<name>/set/   <real val="25.5"/>
```

Attempt	Result
Create a component (POST type, PUT child path)	BadUriErr every variant
Add a history or alarm extension	BadUriErr
Write a proxied point's value	set node is inert

So oBIX owns three things in this workflow: binding BOrd slots, writing DynamicEnum names, and reading history and alarms. Creating the object belongs to REST.

Modbus TCP slave: create the point with its proxy extension

The extension and the register address are what publish the object

	Slave (server)	Client (other direction)
Network	modbusTcpSlave:ModbusTcpSlaveNetwork	modbusTcp:ModbusTcpNetwork
Device	modbusTcpSlave:ModbusSlaveDevice	modbusTcp:ModbusTcpDevice
Proxy ext	ModbusServerNumericProxyExt	ModbusClientNumericProxyExt
Address slot	dataAddress (FlexAddress)	dataAddress (FlexAddress)
Who initiates	external master	station
Polling	none no pollScheduler	devicePollConfig + pollScheduler

Build order that publishes a register

Create the point with PROXY = modbusTcpSlave:ModbusServerNumericProxyExt. The extension is the binding; it cannot be attached after creation.

Set dataAddress on the point (FlexAddress, e.g. Hex:0001) and read it back to confirm the register the master will reach.

cp from a point that already carries the extension is the second route: it works and brings dataAddress, dataType and byteOrder along, so read all three back.

A slave network has no pollScheduler: it answers an external master instead of polling, so the absence of a scheduler is the normal state of this direction. Writes arriving from the master land in the priority array of the station point.

Confirm every protocol from outside the station

Each protocol has one external call that settles the question

Protocol	Outside call	What it proves
BACnet/IP	unicast Who-Is to UDP 47808, then ReadProperty 85	the server answers and the object carries the live value
oBIX	GET /obix/config/ with Basic auth	the export is reachable and typed
Modbus TCP slave	FC03 read from a master off-station	the register maps to the point you think it does

BACnet Who-Is, minimal frame

```
pkt = bvlc(01 00 10 08)        # unicast to <HOST>:47808, not a global broadcast
response  810a 0014 0100 1000 c402 0004 5822 05c4 9100 2124
        device 1112 · maxAPDU 1476 · vendor 36 (Tridium)
```

Send the Who-Is as unicast to the station's own address on UDP 47808; the reply carries the device instance, the maximum APDU and the vendor id.

For ReadProperty, objtype 2 is analogValue and prop 85 is present-value; prop 77 is object-name when you need the identity of the object you reached.

The device instance in the reply is the station's own. Collecting from a field device is the opposite direction and is covered in Part 2.

Which channel performs which step

Two channels, one build each step goes through the one that accepts it

Step	REST v2	oBIX	Why
Create the object	add / cp		typing and creation are REST operations
Rename the component	rename		name follows the identifier you wrote
Bind a BOrd slot		PUT <uri>	BOrd has no REST encoding
Bind a register address	mod dataAddress	PUT	FlexAddress accepts both
Set an enum by name		PUT <enum val=" "/>	REST rejects DynamicEnum values
Read the live value	config	GET	either channel returns the same slot
Write a published value		POST /set/	the export exposes a set node
Batch several edits	/api/batch		one request, applied in order

Reading back is part of the step, not a formality

Every write in this part was confirmed by reading the slot afterwards the write's own response is not the confirmation.

Each protocol then has one external call that settles it: a Who-Is on UDP 47808, a GET on /obix/config, an FC03 read from a Modbus master.

The checklist in VERIFIED.md marks exactly the rows that were reproduced this way: the remaining rows are left blank and carry no claim.

Build order that works against a live station: create over REST, bind the BOrd slot over oBIX, write the identity over REST, then confirm from outside with the protocol's own client. Nothing in this sequence needs a desktop session.